Notes 7: Wildcards

Wildcards

Wildcards are shell patterns. They let one pattern match a bunch of filenames, which is handy when I don't want to type every file name.

`*` matches any number of characters

`*` stands for zero or more characters. So `*.txt` means "anything ending in `.txt`."

Examples:

- `ls *.txt` lists files like `notes.txt` and `todo.txt`, but not `photo.jpg`.
- `rm report*` removes `report1.pdf` and `report_final.docx`, but leaves `my_report.pdf`.
- `cp *.jpg backup/` copies all `.jpg` files into `backup/`, but not `.png` files.

`?` matches exactly one character

One `?` means exactly one character. I always mix this up with `*` and `?` won't grab "any number," it's strictly one.

Examples:

- `ls file?.txt` matches `file1.txt` and `fileA.txt`, but not `file10.txt`.
- `ls ???.sh` matches 3-character `.sh` names like `run.sh` and `fix.sh`, but not `backup.sh`.
- `mv img?.png pics/` moves `img1.png` and `img2.png`, but not `img10.png`.

`[abc]` matches one character from a set

This says "this one spot can be a, b, or c."

Examples:

- `ls [abc]*.txt` matches `apple.txt` and `cherry.txt`, but not `date.txt`.
- `ls file[123].txt` matches `file1.txt`, `file2.txt`, and `file3.txt`, but not `file4.txt`.
- `mv [fhns]*.html components/` moves `footer.html`, `header.html`, `navbar.html`, and `sidebar.html`, but not `index.html`.

`[a-z]` matches one character in a range

Same idea as `[abc]`, but with a range. Works for letters or digits.

Examples:

- `ls [a-f]*` lists files starting with a through f, like `apple.txt` and `fish.txt`, but not `grape.txt`.
- `ls file[0-9].txt` matches `file0.txt` through `file9.txt`, but not `fileA.txt`.
- `rm log[1-5].txt` removes `log1.txt` through `log5.txt`, but leaves `log6.txt`.

`[!abc]` matches one character NOT in the set

The `!` flips it. Match one character that is *not* in the list or range.

Examples:

- `ls [!a]*` — lists files that do not start with `a`, like `banana.txt` and `cherry.txt`, but not `apple.txt`.
- `ls file[!0-9].txt` matches `fileA.txt`, but not `file1.txt`.
- `cp [!.]* backup/` copies files that do not start with a dot, like `notes.txt`, but skips hidden files like `.bashrc`.

Gotcha: the shell expands all of these *before* the command runs. So if nothing matches, some commands will just get the literal pattern (e.g., `ls *.xyz` complains about `*.xyz` not existing).

Brace Expansion

Brace expansion uses `{a,b,c}` to build several names from one pattern. A comma separates siblings, and a `/` goes one level deeper. With `mkdir -p`, I can create an entire directory structure in one command. `-p` creates parent directories, and `-v` prints each directory as it's created.

Example 1: every combination

mkdir -pv assets/{imgs,video}/{large,small}

This creates:

- `assets/imgs/large`
- `assets/imgs/small`
- `assets/video/large`
- `assets/video/small`

Example 2: nested branches

mkdir -pv project/site/{old,new}/{code/{scripts,markup},assets/{imgs,mp3,mp4}}

Inside both `old` and `new`, this creates the following:

- `code/scripts`
- `code/markup`
- `assets/imgs`
- `assets/mp3`
- `assets/mp4`

Example 3: several top-level directories

- mkdir -pv {app,components,data,pages,assets/{css,images},config/deploy}

This makes `app`, `components`, `data`, `pages`, `assets/css`, `assets/images`, and `config/deploy` in one single command.